

Computer Architecture: *Exceptions & Interrupts* *Part 2: Atomics, Nesting, and Faults*

Dr. Colin O'Flynn

Lectures

- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How do you observe the machine running?
- How do we define the machine?
- How does the machine control flow change (planned)?
- How does the machine control flow change (unplanned)?
- **How does the machine control flow change (chaotic unplanned)?**

Part 2-1: Atomic Operations

Interrupting Work

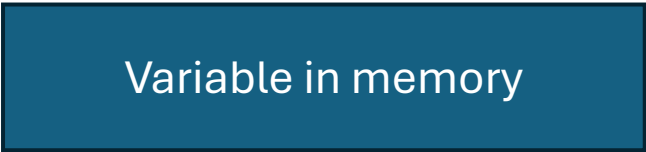
```
counter++;
```

			<code>g_time += 1;</code>
➔ 080001f4	F24001B4	movw r1, #0xB4	
080001f8	F2C10100	movt r1, #0x1000	
080001fc	6808	ldr r0, [r1]	
080001fe	3001	adds r0, #1	
08000200	6008	str r0, [r1]	
<code>main.c — 49</code>			

Race Condition

Main Function

Interrupt



Variable in memory

The diagram illustrates a race condition. It features two labels, 'Main Function' and 'Interrupt', positioned above a central blue rectangular box. The box contains the text 'Variable in memory'. This visualizes a scenario where two different execution paths (Main Function and Interrupt) attempt to access or modify the same memory location simultaneously, leading to a race condition.

Introducing Atomic Operations

- An *atomic operation* is one which is guaranteed to *complete without interruption*
- The previous example of a load, increment, store is a classic example of something which *may* require an atomic operation

Solution #1: No Interrupts!

```
54  
55     __disable_irq();  
56     counter += 1;  
57     __enable_irq();  
58 }
```

```
080001f4  ^/  
          __STATIC_FORCEINLINE void __disable_irq(void)  
          {  
080001f6  [ ] __ASM volatile ("cpsid i" : : : "memory");  
          B672      cpsid i  
          — main.c — 52 —  
          printf("FAIL %d\n", counter);  
          }  
          __disable_irq();  
080001fa  [ ] counter += 1;  
          F24001B4    movw r1, #0xB4  
          F2C10100    movt r1, #0x1000  
          6808      ldr r0, [r1]  
          3001      adds r0, #1  
          6008      str r0, [r1]  
          — cmsis_gcc.h — 947 —  
          Can only be executed in Privileged modes.  
          */  
          __STATIC_FORCEINLINE void __enable_irq(void)  
          {  
08000204  [ ] __ASM volatile ("cpsie i" : : : "memory");  
          B662      cpsie i  
          — main.c — 45 —
```

Arm CPSID / CPSIE

CPSID = Change Processor State - Interrupt Disable

CPSIE = Change Processor State - Interrupt Enable

i = configurable interrupts (NMI, Faults still enabled)

f = faults (too be discussed)

Solution #2: Arm Exclusive Access

```
int atomic_inc(int *p) {  
    int old, status;  
    do {  
        old = __LDREXW(p);  
        status = __STREXW(old + 1, p);  
    } while (status != 0);  
    return old + 1;  
}
```

- Load value + mark “exclusive”
- Attempt store
- Store succeeds only if no interference occurred

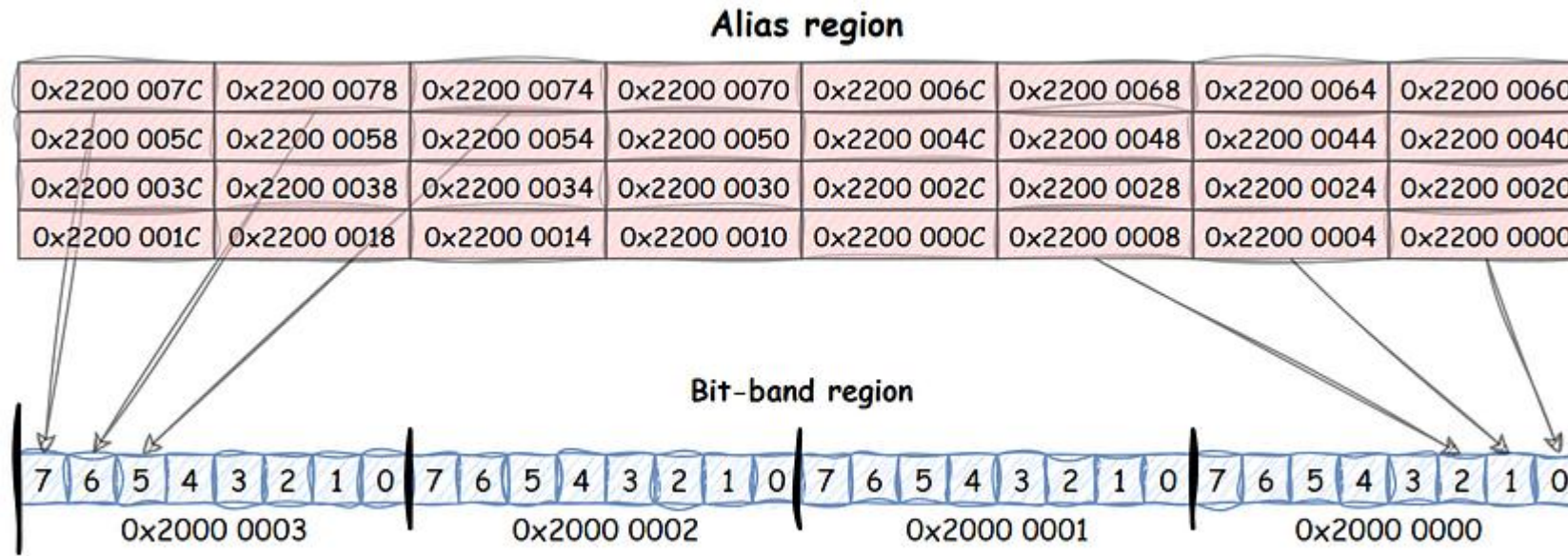
If Interference:

- Store fails
- Software retries

Exclusive Access vs. Disabling Interrupts

- Disabling interrupts can be a risk in a system with time-critical operations
- ***Global*** interrupt disabling can be very impactful on a complex system
- Exclusive access avoids these problems

Solution #3: Bit-Banding



- <https://medium.com/@levinet.nicolai/bit-banding-in-arm-cortex-m-a-brilliant-feature-but-why-was-it-abandoned-139f7ed04685>

Bit Banding

- Each word represents a single bit - this allows writes & clears to affect only one bit
- This is different from devices which often offer a “set” and “clear” bit register (which is more efficient)

Solution #4: Set / Clear Registers (*Peripherals Only*)

8.4.7 GPIO port bit set/reset register (GPIOx_BSRR) (x = A to D and F)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BR[15:0]**: Port x reset I/O pin y (y = 15 to 0)

These bits are write-only. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Resets the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BS[15:0]**: Port x set I/O pin y (y = 15 to 0)

These bits are write-only. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Sets the corresponding ODRx bit

Reminder on bit set/clear in registers

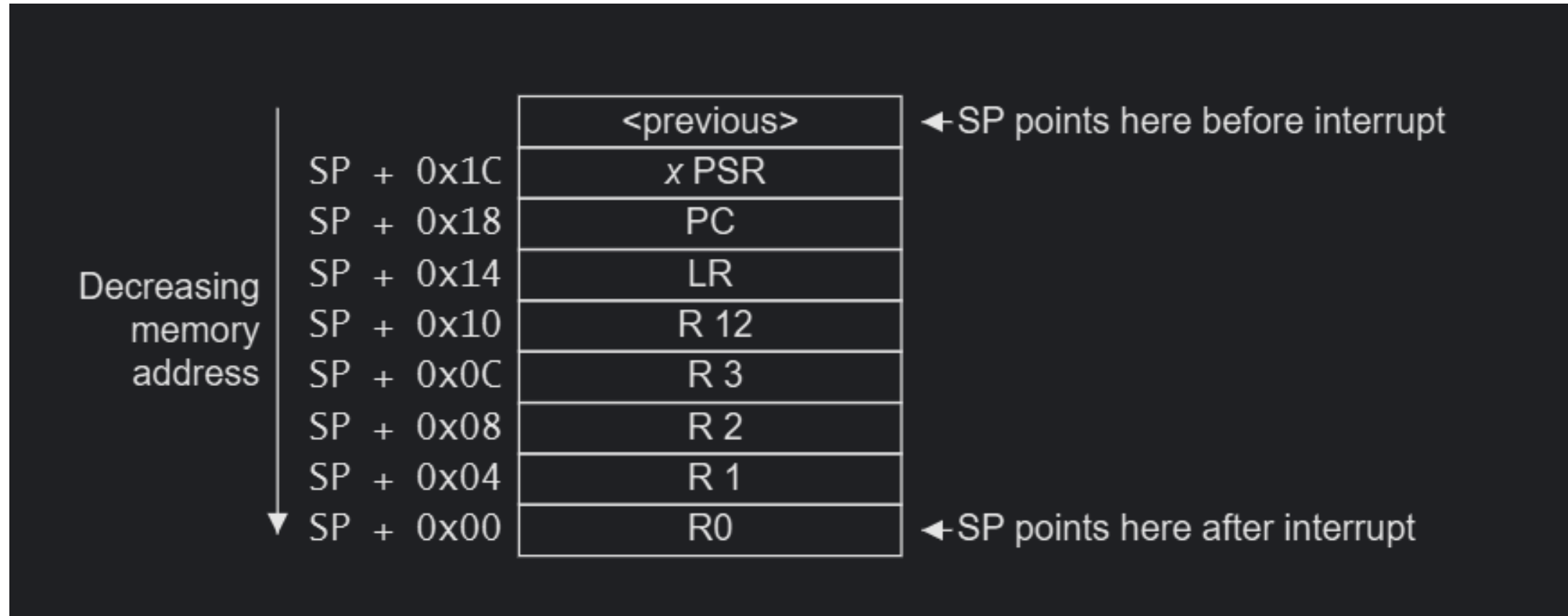
- With registers we can set or clear bits directly
- Remember as Arm is load-store architecture we **always have load and store when dealing with memory**
- Atomic operations are normally referencing atomic operations **on memory**

Atomic Operation Summary

- If disabling interrupts, keep this very short
 - Watch out for function calls would normally avoid! And other things that “could” be slow
- Difficult to debug problems with this as debugging may change timing
- Atomic operations can be needed inside of interrupts if we can nest....

Part 2-2: Nesting

Reminder: Exception Entry / Returns



Endless Interrupt Risks

- Interrupts which re-occur too quickly have a risk of overloading a system.
- Some architectures such as AVR disable global interrupts initially, letting the user decide to re-enable them:

When an interrupt occurs, the global interrupt enable I-bit is cleared and all interrupts are disabled. The user software can write logic one to the I-bit to enable nested interrupts. All enabled interrupts can then interrupt the current interrupt routine. The I-bit is automatically set when a return from interrupt instruction – RETI – is executed.

There are basically two types of interrupts. The first type is triggered by an event that sets the interrupt flag. For these interrupts, the program counter is vectored to the actual interrupt vector in order to execute the interrupt handling routine, and hardware clears the corresponding interrupt flag. Interrupt flags can also be cleared by writing a logic one to the flag bit

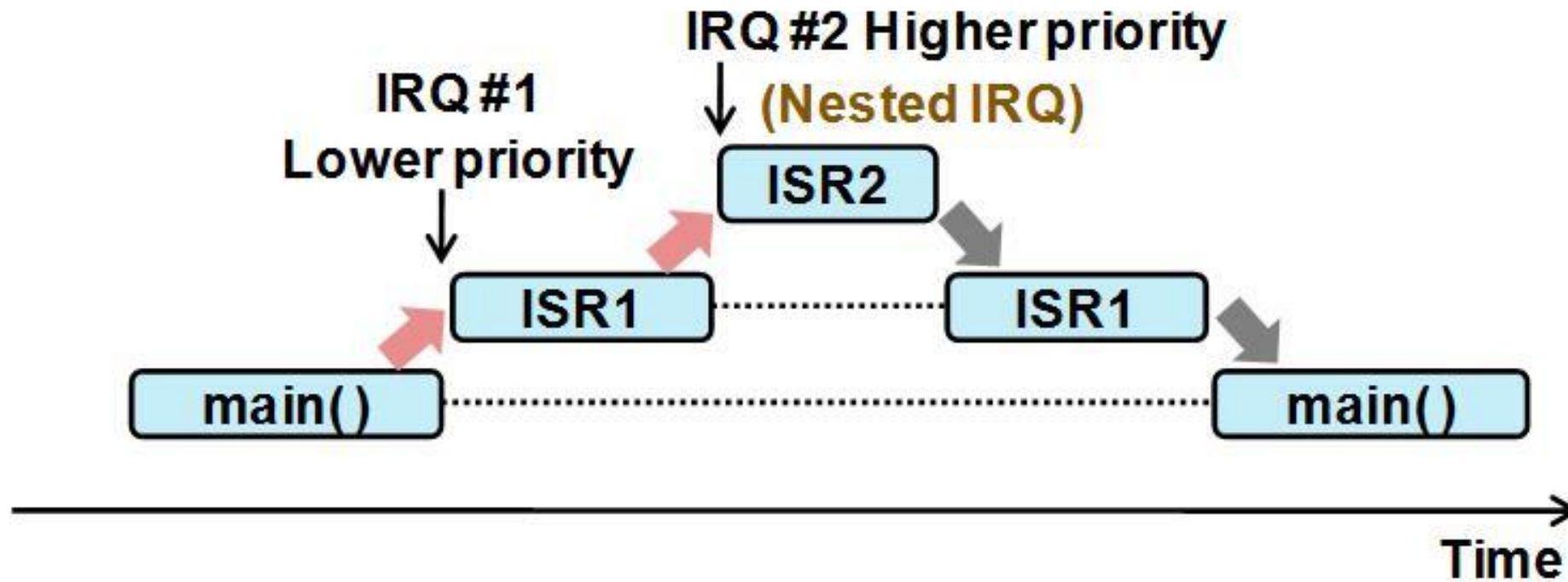
Exception Priorities

- *Most* interrupts have a changeable priority
- Remember lower number = higher priority
- Set priority based on:
 - Interrupt handler speed (short code can have higher priority)
 - System requirements
 - Interrupt frequency

Table 82. STM32F303xB/C/D/E, STM32F358xC and STM32F398xE vector table

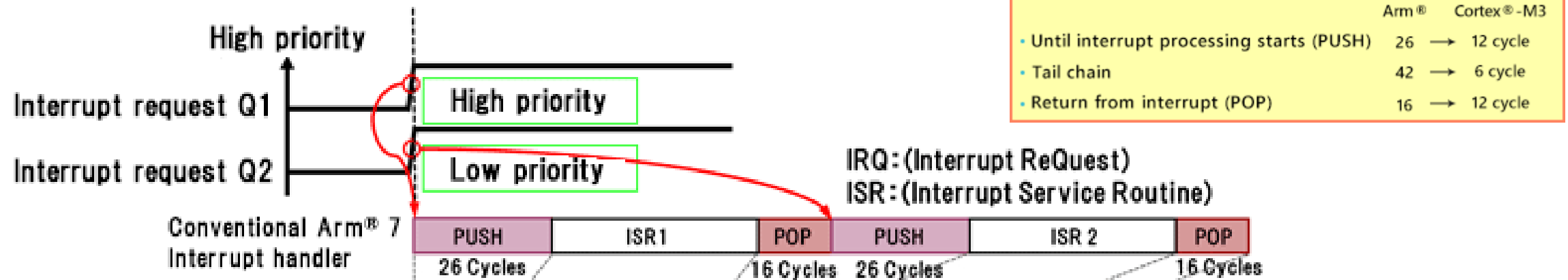
Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	Fixed	Reset	Reset	0x0000 0004
-	-2	Fixed	NMI	Non maskable interrupt. The RCC clock security system (CSS) is linked to the NMI vector.	0x0000 0008
-	-1	Fixed	HardFault	All classes of fault	0x0000 000C
-	0	Settable	MemManage	Memory management	0x0000 0010
-	1	Settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	2	Settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	-	-	-	Reserved	0x0000 001C - 0x0000 0028

Higher Priority Interrupts



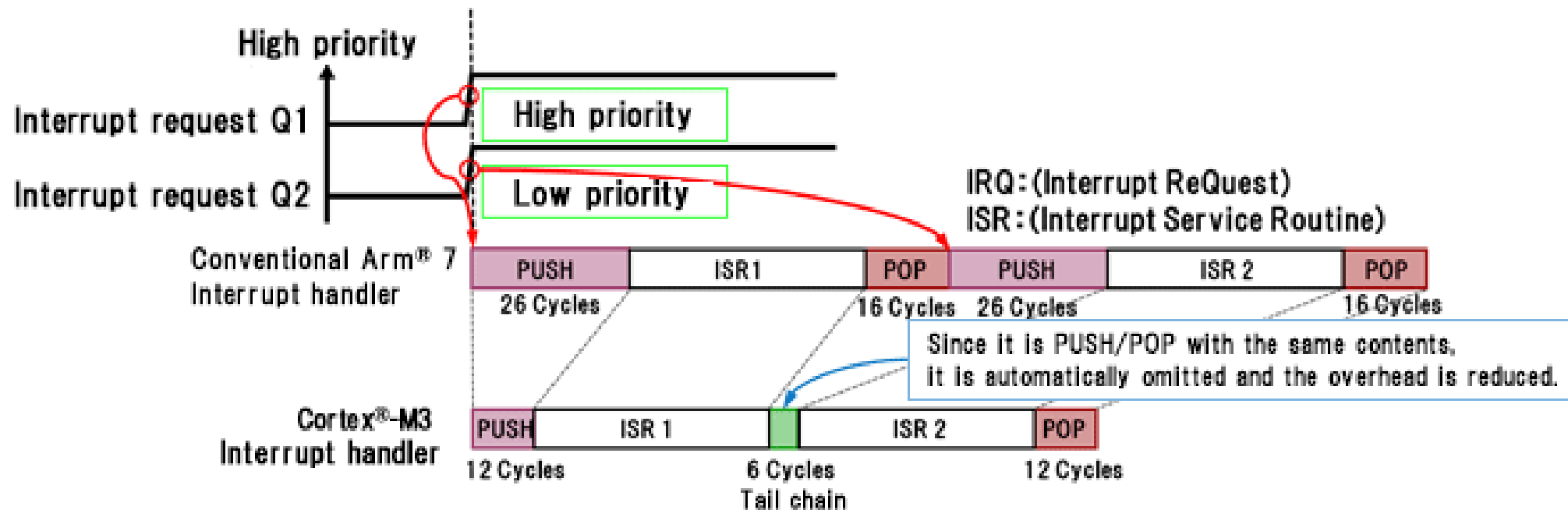
Lower Priority Interrupts & Tail Chaining

- A *lower priority* interrupt that occurs during a *higher priority* interrupt will start executing at the end of the interrupt.



Lower Priority Interrupts & Tail Chaining

- A *lower priority* interrupt that occurs during a *higher priority* interrupt will start executing at the end of the interrupt.



Part 2-3: Faults

Faults

- CPU Needs to deal with issues such as:
 - Invalid instructions
 - Accessing memory that doesn't exist
 - Conflicts on the bus



Faults vs. Aborts

- **Exception:** any hardware-initiated control transfer
- **Interrupt:** asynchronous exception
- **Fault:** exception caused by illegal execution
- **Abort:** fault related to memory access

Cortex-M Faults

- HardFault (catch-all)
- Memory faults
- Bus faults
- Usage faults

the vector table for STM32F303xB/C/D/E, STM32F358xC and STM32F398xE devices.

Table 82. STM32F303xB/C/D/E, STM32F358xC and STM32F398xE vector table

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	Fixed	Reset	Reset	0x0000 0004
-	-2	Fixed	NMI	Non maskable interrupt. The RCC clock security system (CSS) is linked to the NMI vector.	0x0000 0008
-	-1	Fixed	HardFault	All classes of fault	0x0000 000C
-	0	Settable	MemManage	Memory management	0x0000 0010
-	1	Settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	2	Settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	-	-	-	Reserved	0x0000 001C - 0x0000 0028

Hard Fault: The Safety Net

HardFault occurs when:

- A fault is not handled
- Multiple faults happen simultaneously
- System is in an unrecoverable state

Memory Faults: Illegal Access

Memory faults occur when:

- Accessing unmapped memory
- Violating memory protection
- Accessing forbidden regions

General Ideal: Not all addresses are valid memory.

Bus Faults: Access Failed Mid-Flight

Bus faults occur when:

- Address is valid
- Memory system fails
- Peripheral error
- Bus timeout
- External memory failure

General Idea: Address was mapped correctly but some other error

Usage Fault

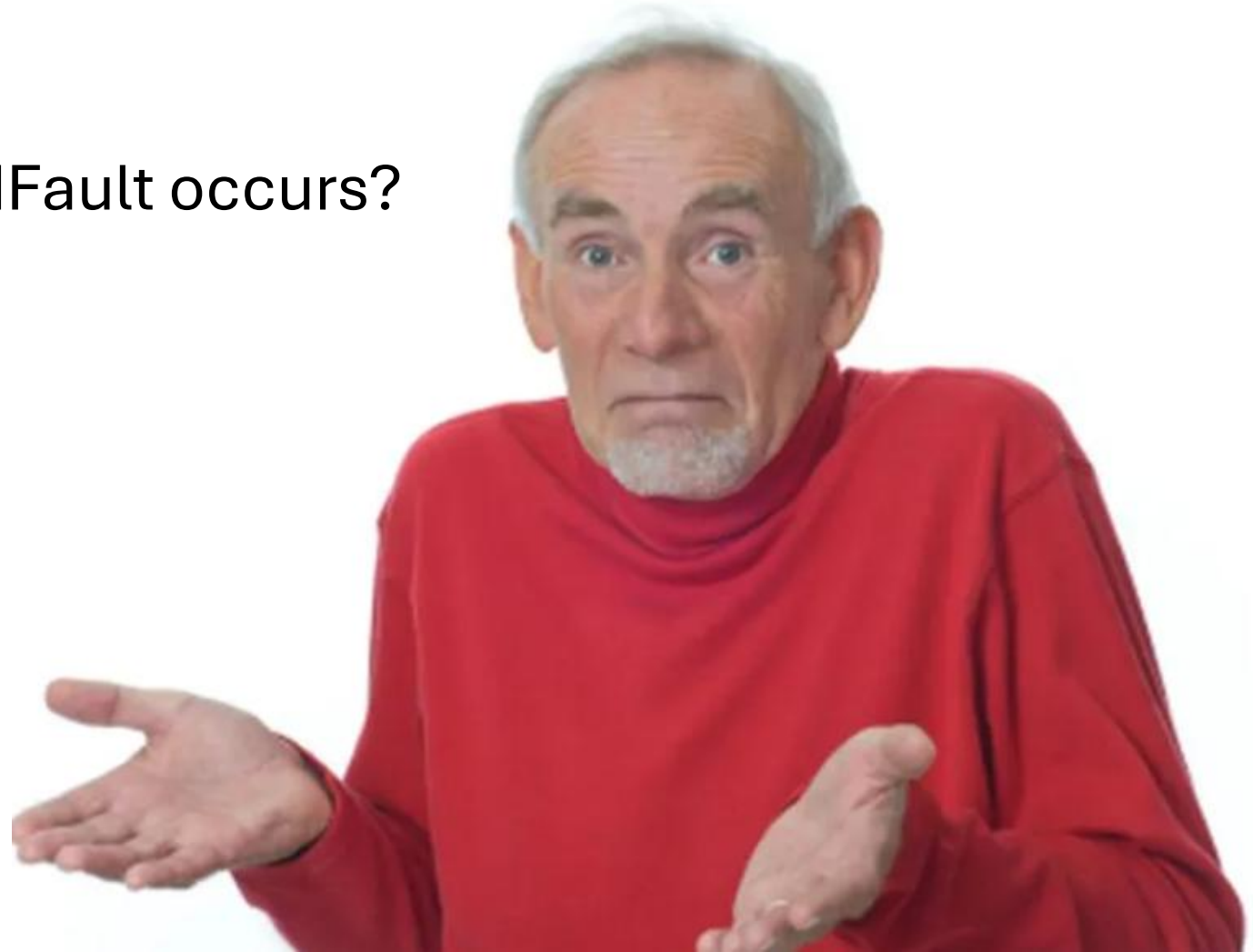
Usage faults occur when:

- Invalid instruction encoding
- Undefined instruction
- Divide by zero (optional)
- Unaligned access (optional)

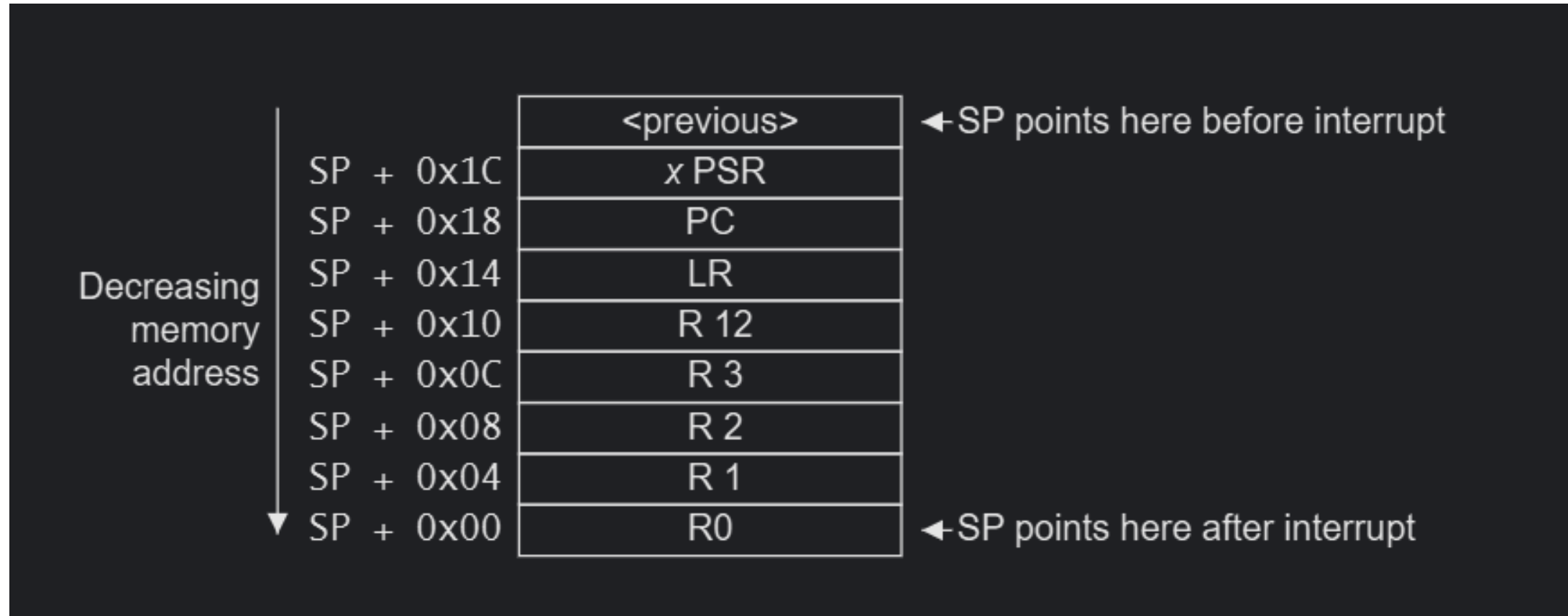
General Ideal: CPU cannot execute meaningless instruction

HardFault Handler

- What do we do when a HardFault occurs?



Remember The Exception Stack



Stack Inspection Saves Us

- The above registers were pushed onto the stack at the moment of the exception.
- We can analyze the exception to understand what was on the stack at that moment in time.

```

43
44 - typedef struct {
45     uint32_t r0, r1, r2, r3;
46     uint32_t r12, lr, pc, xpsr;
47 } stacked_frame_t;
48
49
88 - void hardfault_c(uint32_t *sp) {
89     g_fault_sp = (uint32_t)sp;
90
91     // Copy the stacked registers into
92     g_fault_frame.r0    = sp[0];
93     g_fault_frame.r1    = sp[1];
94     g_fault_frame.r2    = sp[2];
95     g_fault_frame.r3    = sp[3];
96     g_fault_frame.r12   = sp[4];
97     g_fault_frame.lr    = sp[5];
98     g_fault_frame.pc    = sp[6];
99     g_fault_frame.xpsr  = sp[7];

```

Demo: Triggering Faults

```
113
114 - void do_illegal(void) {
115     __asm volatile(".word 0xFFFFFFFF\n");
116 }
117
118 - void jump_illegal(void){
119     void (*fptr)(void) = (void (*)(void))(0xFFFFFFFF);
120     fptr();
121 }
122
123 - void write_illegal(void){
124     uint32_t * ptr = (uint32_t *)(0xFFFFFFFF);
125     *ptr = 12;
126 }
127
```

More information on registers

Several other registers we can inspect – see:

<https://developer.arm.com/documentation/107706/0100/System-exceptions/Fault-exceptions-and-their-causes>

Summary

- Interrupts & Exceptions can introduce *race conditions*
- Using *atomic access* in sections where we cannot have interrupts is important
- Most architectures have some form of *interrupt priority*
 - Implementation details vary
- Faults are an exception used for hard errors
- The HardFault handler on Arm provides considerable additional detail to help you debug.